National University of Computer and Emerging Sciences, (FAST – NUCES) Islamabad

Course: CS-4002 Applied Programming **Term:** MS-CS-Spring-2026

Course Instructor: Bushra Fatima **Deadline:** Sunday, 10 May 2026

Semester Research Project: "NanoDB Architecture & Query Optimizer"

Project Overview: This project is about building a mini database system from scratch, called NanoDB. Think of it like creating a simplified version of MySQL, PostgreSQL, or SQLite. Instead of just writing queries as an end-user, you will be acting as the core systems engineer building the engine that *processes* those queries. You will manage raw memory, design custom data structures, and implement algorithmic optimizations to ensure your database can handle large datasets efficiently.

Disclaimer: This is a graduate-level systems project. Standard Library Containers (e.g., std::vector, std::map, std::stack) are strictly forbidden. You are the architect; build the plumbing yourself.

---

1. Dataset & Benchmarking Standard (TPC-H)

To ensure your engine is evaluated against industry standards, you are required to use the **TPC-H Benchmark Dataset**. TPC-H is a standard decision-support benchmark used universally to test database performance.

You will not be provided with the data files directly. As part of your research and setup, your team must:

1. **Source the Data:** Download or generate a subset of the TPC-H dataset. You must specifically extract and use the following three tables: customer, orders, and lineitem.
2. **Scale Factor:** You must generate or extract exactly **100,000 records** across these tables (e.g., 20,000 customers, 30,000 orders, 50,000 line items).

The Workload File: You must create a **'*queries.txt*'** file containing 50 specific SQL-like string commands that query your TPC-H data (ranging from simple **INSERT** statements to complex multi-condition **SELECT** queries with joins). Your automated Test Runner must read and execute this file during the demo.

---

2. Core Directives & Architectural Hints

A. The Memory Layer (The Pager)

- *Hint:* Your database cannot load a 10GB file into 8GB of RAM. You have a fixed-size raw contiguous block. Manage it.
- *Hint:* When the block is full, who leaves? Implement a cache eviction policy. A Singly Linked List won't cut it for $O(1)$ removals; think bidirectional. If you need a round-robin logging system for transactions, how do you loop without tracking the tail?
- *Constraint:* Serialize and deserialize your memory pages directly to disk via raw File Operations.

B. The Schema & Type Engine

- *Hint:* A row contains an int, a float, and a string. An array expects uniform types. How do you bypass this? Think abstractly. Base classes, virtual tables, and Polymorphism.
- *Hint:* How does the engine know if "Apple" > "Banana" or 10 == 10 without hardcoding if-statements for every type? Operator Overloading is your friend here.
- *Hint:* For metadata (System Catalog: Table names, file paths), $O(N)$ searches are unacceptable. Map them to memory with $O(1)$ collisions resolution. Hashing is mandatory.

C. The Query Parser (The Compiler)

- *Hint:* A user submits: WHERE (Age > 20 AND Salary < 5000) OR Department == "HR". The machine reads a string.
- *Hint:* You need to parse this. Read up on Expression evaluation and Infix to Postfix conversion. Evaluate the resulting mathematical/logical tree using custom Stacks.
- *Constraint:* High-priority admin queries must bypass background user queries. Funnel these through a Priority Queue.

D. The Execution Engine (The Optimizer)

- *Hint:* Sequential Array scans are $O(N)$. Build an index. It must self-correct its height upon insertions/deletions to maintain $O(log\ N)$ lookups. Balance Binary Trees are required.

- *Hint:* Multi-way joins are expensive. If the user queries Table A JOIN Table B JOIN Table C, what is the cheapest execution path?
- *Hint:* Map the tables as nodes and join-costs as edges (Graph Representation). Find the cheapest path without cycles. Minimal Spanning Trees will be your query optimizer.

---

3. Deliverables & Version Control

By the final deadline, you must submit their work following these strict version control and packaging guidelines:

A. GitHub Repository (Mandatory): Your codebase must be maintained in a public GitHub repository.

- We expect to see a continuous history of commits. A repository with a single massive commit at the end will not be acceptable.
- Use a proper ***'.gitignore'*** file (do not push compiled .exe files or the raw TPC-H datasets to GitHub).

B. Project Archive: A .zip file uploaded to the GCR containing:

- The complete C++ Source Code (cleanly separated into .h and .cpp files).
- A Build Script (Makefile or CMakeLists.txt) to compile your engine via the command line.
- The Automated Test Runner: A specific executable script or file (e.g., *test_runner.cpp* or *run_tests.sh*) that automatically processes your *queries.txt* Workload File.
- A *README.md* containing a link to your GitHub repo and explicit instructions on how to compile and execute the Test Runner.

C. The Research Report (PDF): A formal 5-8 page technical paper documenting:

- Mathematical time/space complexity proofs for your custom data structures (Buffer Pool, Indexer, Parser).
- Empirical benchmarking graphs (Execution time vs. Data Size).
- Memory profiling analysis (LRU cache page-fault rates).

4. Live Demo & Evaluation Protocol

Evaluation will be conducted via a strict 15-minute live demonstration. The evaluation will heavily rely on your system's ability to log its internal decision-making processes.

The Automated Run & Log Evaluation:

At the start of the demo, the evaluator will instruct you to execute your Test Runner file. Your engine must process your ***'queries.txt'*** Workload File and output a highly detailed ***'nanodb_execution.log'*** file. The evaluator will review this log in real-time to verify:

- Cache eviction events (e.g., *[LOG] Page 42 evicted via LRU, written to disk*).
- Query parsing steps (e.g., *[LOG] Infix "c_acctbal > 5000" converted to Postfix "c_acctbal 5000 >"*).
- Optimizer decisions (e.g., *[LOG] Multi-table join routed via MST: customer -> orders -> lineitem*).

Specific Demo Execution Steps:

Alongside the log review, you will execute the following live test cases using the TPC-H schema:

- **Test Case A (The Parser & Evaluator):** Input a complex string: SELECT WHERE (c_acctbal > 5000 AND c_mktsegment == "BUILDING") OR c_nationkey == 15. Your engine must print the generated Postfix expression from your custom Stack and output the correct filtered rows.
- **Test Case B (The Index Optimizer):** Execute a search for a specific c_custkey on your 100,000 records. First, force the engine to use a sequential Array scan and print the execution time. Second, run the exact same query using your custom Balanced Binary Tree index. The time reduction must be visibly printed to the console.
- **Test Case C (The Join Optimizer):** Execute a 3-table join (customer JOIN orders JOIN lineitem). Print the generated Minimal Spanning Tree (MST) path your graph-based optimizer chose before outputting the joined data.
- **Test Case D (The Memory Stress Test):** We will artificially restrict your Buffer Pool array to only 50 memory pages. You will then run a query that requires scanning 5,000 lineitem records. Your engine must print the total number of times the Doubly Linked List LRU cache had to evict an old page to disk.
- **Test Case E (Priority Queue Concurrency):** We will simulate a flooded system by submitting 50 standard SELECT queries into your queue, immediately followed by

an admin-level UPDATE transaction to a customer's balance. Your Priority Queue must intercept and execute the admin transaction before finishing the background reads.

- **Test Case F (Deep Expression Tree Edge Case**): We will input a deeply nested and mathematically complex query: SELECT WHERE ((o_totalprice * 1.5) > 100000 AND (o_custkey % 2 == 0)) OR (o_orderstatus != "O"). Your system must correctly evaluate the operator precedence and mathematical mutations using your Tree/Stack parsers without crashing.
- **Test Case G (Durability & Persistence):** You will insert 5 new records into the customer table. You will then completely terminate the NanoDB program. Upon rebooting the engine, you must successfully query those 5 new records, proving your File Operations successfully serialized the custom memory pages to disk before shutdown.

The Viva Defense:

Following the demo, the evaluator will randomly select blocks of your data structure implementations. You must explain the pointer arithmetic and algorithmic logic on the spot.

---

5. Grading & Penalty Box

- **The Standard Template Library (STL) Ban:** The use of ANY C++ STL container (std::vector, std::list, std::map, std::stack, std::queue, std::set, etc.) or algorithm (std::sort, std::find) will result in an immediate -100% deduction for that specific architectural section. You must build the tools you use.
- **Memory Leaks:** Failure to properly delete dynamically allocated memory (verified via tools like Valgrind) will result in a flat -15 point deduction from the final grade.
- **Plagiarism & Academic Integrity:** Inability to explain your own codebase during the Viva, missing execution logs, or discrepancies in your GitHub commit history (e.g., massive copy-pasted single commits) will be treated as an academic integrity violation, resulting in an F grade for the project and course as per university policy.

Research & Benchmarking Deliverables

Do not just submit code. You must prove your engine works mathematically and empirically.

1. **Complexity Analysis:** Document the theoretical time and space complexity for your specific implementations of the Buffer Pool, Indexer, and Query Parser.
2. **Empirical Benchmarking:** Plot execution times for 1K, 10K, and 100K record insertions and retrievals. Compare your Balanced Tree Index vs. an unindexed sequential scan.
3. **Memory Profiling:** Force a bottleneck. Allocate only 50 memory pages for 10,000 records. Graph your LRU cache's page-fault rate.

Grading Rubric: NanoDB Architecture & Query Optimizer - Total Points: 100

---

1. Architecture & OOP Foundations (20 Points)

This section evaluates the structural design of the database, focusing on memory management and object-oriented principles.

Criteria	*Excellent (Full Points)*	*Poor (Zero/Low Points)*	*Points*
Buffer Pool & Memory	Implements custom fixed-size memory array via raw pointers. Flawless pointer arithmetic without leaks.	Relies on dynamic resizing without underlying manual array management.	**10**
Polymorphic Types	Elegant use of base and derived classes for heterogeneous row data. Clean operator overloading for all data types.	Hardcoded if-statements for type checking. No use of virtual functions.	**5**
File I/O	Direct and efficient binary serialization/deserialization of pages to disk.	Storing data as inefficient plain text or failing to persist data between runs.	**5**

2. Core Data Structures & Execution (30 Points)

This section assesses the implementation of fundamental data structures and their theoretical time complexities.

Criteria	*Excellent (Full Points)*	*Poor (Zero/Low Points)*	*Points*
LRU Cache (Linked Lists)	Implements a robust O (1) eviction policy using a custom Doubly Linked List and proper pointer manipulation.	Uses O(N) arrays for cache management or fails to evict old pages.	**10**
Query Parser (Stacks/Queues)	Flawless Infix to Postfix conversion using custom Stacks. Accurate mathematical/logical tree evaluation.	Fails to handle operator precedence or nested parentheses.	**10**
Indexing (Balanced Trees)	Custom AVL or Red-Black Tree implemented. Auto-balances correctly to guarantee O (log N) search time.	Implements a standard BST that degrades to O(N) or uses sequential array scans.	**10**

3. Advanced Algorithms & Optimization (20 Points)

This section grades the high-level routing and metadata management.

Criteria	*Excellent (Full Points)*	*Poor (Zero/Low Points)*	*Points*
System Catalog (Hashing)	Custom Hash Map with efficient collision resolution (e.g., chaining via Linked Lists). Achieves O (1) lookups.	High collision rates, poor hash function, or degradation to O(N) search.	**10**
Query Optimizer (Graphs & MST)	Accurately models multi-table joins as a Graph. Correctly applies Kruskal's or Prim's algorithm to find the Minimal Spanning Tree.	Brute-forces multi-table joins. Fails to implement Graph representation.	**10**

4. Research, Benchmarking & Complexity Analysis (30 Points)

Criteria	*Excellent (Full Points)*	*Poor (Zero/Low Points)*	*Points*
Complexity Documentation	Rigorous mathematical proofs of time and space complexities for all major structures and algorithms.	Missing or mathematically incorrect O-notation analysis.	**10**
Empirical Benchmarking	Comprehensive graphs comparing execution times (1K vs 10K vs 100K records). Compares indexed vs. unindexed queries.	No graphs provided, or testing size is too small to show algorithmic scaling.	**10**
Memory Profiling	In-depth analysis of the LRU cache page-fault rates under constrained memory limits.	Ignores memory constraints or fails to profile the eviction policy.	**10**

Critical Deductions & Penalty Box

- **The Standard Template Library (STL) Ban:** The use of **ANY** C++ STL container (std::vector, std::list, std::map, std::stack, std::queue, std::set, etc.) or algorithm (std::sort, std::find) will result in an immediate **-100% deduction for that specific rubric section**. You must build the tools you use.
- **Memory Leaks:** Failure to properly delete dynamically allocated memory (verified via tools like Valgrind) will result in a flat **-15 point deduction** from the final grade.
- **Plagiarism & Academic Integrity:** Plagiarism in the project will result in an F grade in the course. This includes copying code from peers, the internet, or passing off heavily generated LLM code as your own without deep understanding. You will be asked to defend your architectural decisions in a viva voce.
